POT WATCH

Node Health Monitor & Staking Analytics Suite

Live watcher engine + on-chain AlertRegistry contract + zero-backend dashboard.

Alerts are delivered to Discord/Telegram AND verifiably recorded on the Portaldot V3 testnet.

`dashboard: iscyman.github.io/pot-watch` · `repo: github.com/iscyman/pot-watch`

Week 2 check-in deck — 22 Oct 2026 · solo build · MIT license

The problem

Operators & delegators are blind

Portaldot node health (uptime, era behaviour, finality) is only visible by running your own tooling. No live, public picture exists.

Alerts live off-chain — unprovable

A Discord ping can be faked, lost or edited. There is no tamper-evident record that an alert condition actually happened, when, and who saw it.

V3 hides a brutal fee wall

Deploying the same contract costs ~289 tPOT via the EVM/MetaMask surface but 0.36 tPOT natively (~800x). Faucet-funded builders silently hit a wall.

Why Portaldot is essential

POT Watch is not a generic monitor ported onto a chain — it is built from Portaldot V3 specifics:

■ Monitors the real security set

BABE authorship digests, era/session rotation and validator
activity of the exact set securing the network — on mainnet AND the V3 testnet.

■ Uses BOTH execution surfaces on purpose

Writes go through the native Revive pallet (cheap, deposit-based);
reads and tooling go through the EVM JSON-RPC (eth_call, eth_getCode).

■ Makes alerts part of chain history

AlertRegistry lives at 0xeae1…429d on chainId 420420777 —
registrations and fires are finalized extrinsics anyone can audit.

The product — three pillars

WATCHER

TypeScript engine, 24/7.
Streams blocks/eras/sessions,
decodes BABE authors,
per-validator uptime,
composite health score 0–100,
Discord/Telegram delivery.
self-check suite 9/9.

CONTRACT

AlertRegistry (Solidity 0.8.24,
london). Kinds: uptime, era
rotation, slash, large transfer.
Channels, thresholds,
permissionless recordFire with
per-block cooldown, fire stats.
6/6 unit tests.

DASHBOARD

Zero backend, zero deps.
Browser talks raw JSON-RPC
to both surfaces; hand-rolled
SCALE/twox128/ss58/BABE
decoding. Live block stream,
validator table, health gauge,
on-chain alert center.

One loop closes it all: watcher detects → alert delivered → alert provable on-chain → dashboard shows it live.

Proof — live on the V3 testnet

Contract (AlertRegistry)	`0xeae1fb99ac651fd8a8bc23fb65dc51acb925429d`
Deploy tx (finalized)	`0xcbf48e01dd166a0fd73b08942718049c87444652c82ad4cc11b7899f020fd8d3`
Deploy block / route	`133258 · Revive.instantiate_with_code (native surface)`
Interaction 1 (blk 133316)	`registerAlert ValidatorUptime→Discord 90.00% · 0x0fedf0f6…`
Interaction 2 (blk 133319)	`registerAlert EraRotation→Telegram · 0x38b85ab4…`
Interaction 3 (blk 133322)	`recordFire #1 · 0x39b60c92…`
Events decoded	`AlertRegistered x2 + AlertFired x1 (from revive.ContractEmitted)`
EVM-side verification	`eth_getCode 3,429 B · eth_call reads live (activeAlertCount = 2)`
Health score (live)	`91 / 100 — EXCELLENT (finality, availability, block time, HHI)`
Evidence file	`contracts/deployment.json — machine-readable, in repo`

Every number above is reproducible from the explorer link in the repo README.

Ecosystem contribution: the dual fee model

Same contract, same chain:

EVM / MetaMask surface

Native Revive surface (ours)

What we mapped & open-sourced:

- weight↔gas scaling, storage deposits, ED rules
- why native writes are invisible to EVM log indexes
  (events live in revive.ContractEmitted — we decode them)
- working deploy/interact/decode scripts for any team
- full write-up: docs/DEPLOYMENT.md

≈ 800× cheaper — faucet-scale teams can ship contracts too. That tooling is public in the repo.

Judging note: this is original chain research produced during the hackathon, useful beyond POT Watch.

Roadmap to the Nov 6 freeze

DONE Oct 17–22

watcher engine · AlertRegistry deployed & exercised · fee-model research · zero-backend dashboard live · 6 commits, 35 files

NOW Oct 23–25

autonomous alert loop: watcher fires recordFire on-chain by itself (check-in + this deck + demo video)

NEXT Oct 26–31

security analytics: slash-event + large-transfer detection feeding the SlashEvent / LargeTransfer kinds · Telegram channel · office hours

FINAL Nov 1–5

uptime history 24h/7d · polished deck + ≤3-min video · final submission before the Nov 6, 04:00 UTC freeze

Why support POT Watch after the hackathon

Open source & maintained

MIT licensed, single-purpose modules, zero heavy dependencies — cheap to run, easy to audit, and already documented end-to-end (README, ARCHITECTURE, DEPLOYMENT, CHECKIN).

A product, not a demo

Alerting-as-a-service: delegators subscribe to uptime SLAs for their validators; node operators prove response times on-chain. The registry contract is the settlement layer for that market.

Immediate ecosystem value

The native deploy tooling unblocks every faucet-funded team on V3 today; the ContractEmitted decode path documents behaviour the explorer does not yet surface.

Candidate for the Portaldot ecosystem Builder support program — we intend to keep shipping.

Live today: dashboard, contract, watcher, tooling. · github.com/iscyman/pot-watch